

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №1
по дисциплине: Архитектура вычислительных систем
тема: «**Разработка программ на ассемблере.**
Работа с отладчиком x32dbg, пакетом nasm32»

Выполнил: студент группы ВТ-231
Дроботенко Я. Е.
Проверили:
Осипов О.В.
Бондаренко Т.В.

Белгород 2025

Цель работы: получить навыки создания простейших ассемблерных программ с использованием пакета `masm32` и научиться пользоваться отладчиком `x32dbg`.

Примечание: на лекции было сказано о том, что при выполнении лабораторной работы можно использовать операционную систему линукс. Поэтому, так как я привык работать именно в ней, лабораторная будет выполнена с использованием возможностей этой системы — в частности компилятор `masm`, а так же отладчик `gdb`

Задание по варианту №5:

Исходный код на MASM32:

```
5. .DATA
    str1 DB "Lfngth: ", 0
    len DW 0
    mas DD 8 DUP(1)
    x DQ 1.0, 2.0
    ten DT 3000000000
.CODE

START:
    XOR ESI, ESI
    ADD ESI, 8
    MOV str1[ESI], '9'
    DEC str1[1]
END START
```

Исходный код программы на NASM для Linux:

```
section .data
    str1 db "Lfngth: ", 0 ; Строка с нулевым терминатором
    len dw 0 ; 16-битная переменная, равна 0
    mas dd 1, 1, 1, 1, 1, 1, 1, 1 ; Массив из 8 двойных слов, каждое равно 1
    x dq 1.0, 2.0 ; Два 64-битных числа с плавающей точкой
    ten dt 3000000000.0 ; 80-битное число с плавающей точкой (исправлено)

section .text
    global _start ; Точка входа для Linux

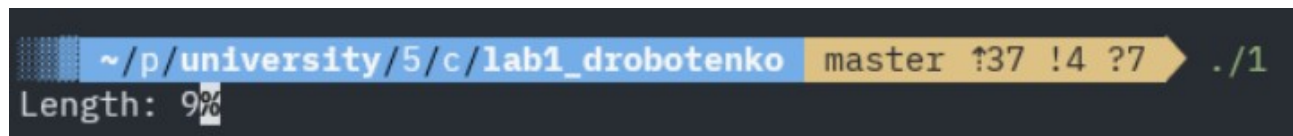
_start:
    ; Модификация строки
    xor esi, esi ; Обнуляем ESI
    add esi, 8 ; Устанавливаем ESI в 8
    mov byte [str1 + esi], '9' ; Записываем символ '9' в str1[8]
    dec byte [str1 + 1] ; Уменьшаем ASCII-код str1[1] ('f' -> 'e')
    ; Вывод строки на экран
    mov eax, 4 ; Номер системного вызова write
    mov ebx, 1 ; Дескриптор файла 1 (stdout)
    mov ecx, str1 ; Адрес строки
    mov edx, 9 ; Длина строки (включая '9')
    int 0x80 ; Вызов ядра
    ; Завершение программы
    mov eax, 1 ; Номер системного вызова exit
    mov ebx, 0 ; Код возврата 0
    int 0x80 ; Вызов ядра gdb
```

Компиляция и запуск:

Для компиляции и запуска программы на Linux с использованием NASM и LD используются следующие команды bash:

```
nasm -f elf32 -g -F dwarf lab1.asm -o lab1.o
ld -m elf_i386 lab1.o -o lab1
./lab1
```

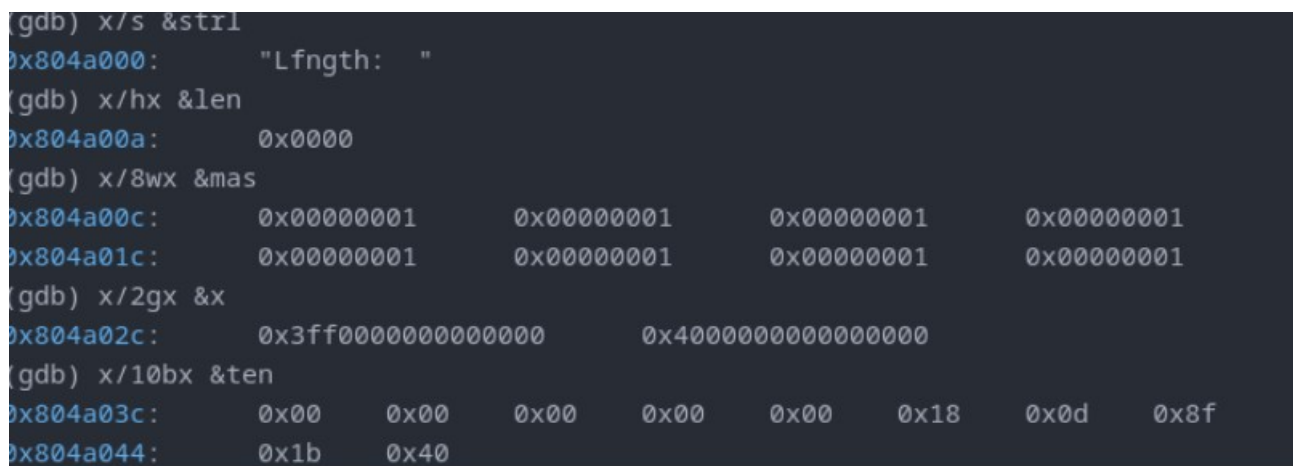
Результат выполнения



```
~/p/university/5/c/lab1_drobotenko master ↑37 !4 ?7 ./1
Length: 9%
```

Код выводит строку согласно одному из подпунктов пункта 4

Анализ сегмента данных с помощью GDB:



```
(gdb) x/s &str1
0x804a000: "Lfngth: "
(gdb) x/hx &len
0x804a00a: 0x0000
(gdb) x/8wx &mas
0x804a00c: 0x00000001 0x00000001 0x00000001 0x00000001
0x804a01c: 0x00000001 0x00000001 0x00000001 0x00000001
(gdb) x/2gx &x
0x804a02c: 0x3ff0000000000000 0x4000000000000000
(gdb) x/10bx &ten
0x804a03c: 0x00 0x00 0x00 0x00 0x00 0x18 0x0d 0x8f
0x804a044: 0x1b 0x40
```

str1 — адрес 0x804a000, содержит строку "Lfngth: " с нулевым терминатором, размер 10 байт.

len — адрес 0x804a00a, значение 0, размер 2 байта (16-битное слово).

mas — адрес 0x804a00c, массив из 8 элементов, каждый равен 1, размер 32 байта (8 × 4 байта).

x — адрес 0x804a02c, содержит два 64-битных числа с плавающей точкой: 1.0 и 2.0, размер 16 байт.

ten — адрес 0x804a03c, 80-битное число с плавающей точкой со значением 300000000.0, размер 10 байт.

Дамп памяти:

```
(gdb) x /34xb 0x0804a000
0x804a000: 0x0d 0x0a 0x5f 0x5f 0x5f 0x5f 0x00 0x05
0x804a008: 0x00 0x90 0x90 0x90 0x00 0x00 0x00 0x00
0x804a010: 0x00 0x00 0x00 0x00 0x00 0x80 0x31 0x40
0x804a018: 0x00 0x00 0x00 0x00 0x00 0x60 0x09 0xb3
0x804a020: 0x10 0x40
```

Адрес	Шестнадцатеричное	Интерпретация
0x0804a000	0x4c 0x66 0x6e 0x67 0x74 0x68 0x3a 0x20	Строка "Lfngh: " (начало strl)
0x0804a008	0x20 0x00 0x00 0x00	Пробел + нулевой терминатор (strl завершена), len = 0x0000 (2 байта), выравнивание до 4-байтной границы
0x804a00c	0x01 0x00 0x00 0x00 ... (8 раз)	Массив mas: 8 значений 1 (каждое — 32-битное, little-endian)
0x804a02c	0x00 0x00 0x00 0x00 0x00 0x00 0xf0 0x3f	x[0] = 1.0 (double в little-endian)
0x804a034	0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x40	x[1] = 2.0 (double в little-endian)
0x804a03c	0x00 0x00 0x00 0x00 0x00 0x18 0x0d 0x8f 0x1b 0x40	ten = 300000000.0 (80-битное число с плавающей точкой, x87 extended precision)

Пошаговая трассировка программы:

Запускаем программу в GDB и пошагово выполняем инструкции, наблюдая за изменениями регистров и памяти

```
Breakpoint 1, 0x08049000 in _start ()
(gdb) stepi
0x08049002 in _start ()
(gdb) info registers
eax            0x0            0
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xffffcd20      0xffffcd20
ebp            0x0            0x0
esi            0x0            0
edi            0x0            0
eip            0x8049002        0x8049002 <_start+2>
eflags         0x246          [ PF ZF IF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0            0
gs             0x0            0
(gdb) stepi
0x08049005 in _start ()
(gdb) info registers
eax            0x0            0
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xffffcd20      0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x8049005        0x8049005 <_start+5>
eflags         0x202          [ IF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0            0
gs             0x0            0
```

```

(gdb) info registers
eax            0x0            0
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xffffcd20     0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x804900c       0x804900c <_start+12>
eflags         0x202          [ IF ]
cs             0x23           35
ss             0x2b           43
ds             0x2b           43
es             0x2b           43
fs             0x0            0
gs             0x0            0
(gdb) stepi
0x08049012 in _start ()
(gdb) info registers
eax            0x0            0
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xffffcd20     0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x8049012       0x8049012 <_start+18>
eflags         0x206          [ PF IF ]
cs             0x23           35
ss             0x2b           43
ds             0x2b           43
es             0x2b           43
fs             0x0            0
gs             0x0            0
(gdb) stepi
0x08049017 in _start ()
(gdb) info registers
eax            0x4            4
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xffffcd20     0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x8049017       0x8049017 <_start+23>
eflags         0x206          [ PF IF ]
cs             0x23           35

```

```

eip          0x8049017      0x8049017 <_start+23>
eflags       0x206         [ PF IF ]
cs           0x23          35
ss           0x2b          43
ds           0x2b          43
es           0x2b          43
fs           0x0           0
gs           0x0           0
(gdb) stepi
0x0804901c in _start ()
(gdb) info registers
eax          0x4           4
ecx          0x0           0
edx          0x0           0
ebx          0x1           1
esp          0xffffcd20    0xffffcd20
ebp          0x0           0x0
esi          0x8           8
edi          0x0           0
eip          0x804901c     0x804901c <_start+28>
eflags       0x206         [ PF IF ]
cs           0x23          35
ss           0x2b          43
ds           0x2b          43
es           0x2b          43
fs           0x0           0
gs           0x0           0
(gdb) stepi
0x08049021 in _start ()
(gdb) info registers
eax          0x4           4
ecx          0x804a000     134520832
edx          0x0           0
ebx          0x1           1
esp          0xffffcd20    0xffffcd20
ebp          0x0           0x0
esi          0x8           8
edi          0x0           0
eip          0x8049021     0x8049021 <_start+33>
eflags       0x206         [ PF IF ]
cs           0x23          35
ss           0x2b          43
ds           0x2b          43
es           0x2b          43
fs           0x0           0
gs           0x0           0
(gdb) stepi

```



```

(gdb) stepi
0x08049026 in _start ()
(gdb) info registers
eax            0x4                4
ecx            0x804a000        134520832
edx            0x9              9
ebx            0x1              1
esp            0xffffcd20       0xffffcd20
ebp            0x0              0x0
esi            0x8              8
edi            0x0              0
eip            0x8049026        0x8049026 <_start+38>
eflags         0x206            [ PF IF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0              0
gs             0x0              0
(gdb) stepi
Length: 90x08049028 in _start ()
(gdb) info registers
eax            0x9              9
ecx            0x804a000        134520832
edx            0x9              9
ebx            0x1              1
esp            0xffffcd20       0xffffcd20
ebp            0x0              0x0
esi            0x8              8
edi            0x0              0
eip            0x8049028        0x8049028 <_start+40>
eflags         0x206            [ PF IF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0              0
gs             0x0              0

```

```

(gdb) stepi
0x0804902d in _start ()
(gdb) info registers
eax            0x1            1
ecx            0x804a000      134520832
edx            0x9           9
ebx            0x1            1
esp            0xffffcd20     0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x804902d      0x804902d <_start+45>
eflags         0x206          [ PF IF ]
cs             0x23           35
ss             0x2b           43
ds             0x2b           43
es             0x2b           43
fs             0x0            0
gs             0x0            0
(gdb) stepi
0x08049032 in _start ()
(gdb) info registers
eax            0x1            1
ecx            0x804a000      134520832
edx            0x9           9
ebx            0x0            0
esp            0xffffcd20     0xffffcd20
ebp            0x0            0x0
esi            0x8            8
edi            0x0            0
eip            0x8049032      0x8049032 <_start+50>
eflags         0x206          [ PF IF ]
cs             0x23           35
ss             0x2b           43
ds             0x2b           43
es             0x2b           43
fs             0x0            0
gs             0x0            0
(gdb) stepi
[Inferior 1 (process 59892) exited normally]

```


Выводы

В ходе лабораторной работы была написана программа на ассемблере NASM для платформы Linux, которая выполняет арифметические операции (сложение и деление) и сохраняет результат в переменную. Программа была успешно скомпилирована и запущена. С помощью отладчика GDB был проведен анализ сегмента данных и пошаговая трассировка программы, в ходе которой было подтверждено корректное выполнение всех инструкций. Получены навыки работы с отладчиком и анализа памяти.